Organizational Knowledge & Decision Intelligence RAG

Updated Project Blueprint - v2

Advanced RAG + Backend + AWS + Full-Stack Portfolio System

Status	Decision
Architecture planning	Substantially complete
Primary product direction	Enterprise-style organizational knowledge + focused decision provenance
Cloud direction	Cost-conscious AWS portfolio deployment on EC2
Current corpus candidates	GitLab vs. Kubernetes; final selection pending validation
Next milestone	Corpus validation, repository startup, baseline dense retrieval

Core thesis
Enterprise RAG is difficult because organizational information changes over time, sources have different levels of authority, and documents can conflict. The system will model those properties explicitly instead of asking the final LLM to resolve them implicitly.

Contents

1. Project summary and status
2. Product concept and problem thesis
3. Scope: Phase 1 and Phase 2
4. Primary user outcomes and query classes
5. Corpus and data strategy
6. Open-source positioning
7. Engineering principles and 80/20 priorities
8. Target system architecture
9. Technology stack
10. Ingestion and parsing
11. Chunking and enrichment
12. Embeddings
13. PostgreSQL/pgvector data model
14. Retrieval architecture
15. Temporal and authority-aware retrieval
16. Conflict detection
17. Agentic orchestration
18. Generation, citations, and refusal
19. Evaluation and experimentation
20. Observability
21. Backend engineering and APIs
22. Asynchronous ingestion
23. AWS deployment architecture
24. CI/CD and Infrastructure as Code
25. Frontend roadmap
26. Testing and production hardening
27. Cost and operational constraints
28. Explicit non-goals and deferred technologies
29. Development roadmap
30. Definition of done
31. Portfolio and interview positioning
32. Remaining planning/readiness gaps

1. Project summary and status

Build a production-style Organizational Knowledge and Decision Intelligence platform using advanced Retrieval-Augmented Generation (RAG) over a real, public organization corpus. The project is designed to demonstrate modern information retrieval, agentic AI workflows, backend engineering, AWS/cloud deployment, CI/CD, and a polished full-stack product without adding infrastructure that the problem does not justify.

Planning status
The architecture and technology direction are sufficiently defined to start development. Remaining work is pre-build readiness: final corpus validation, MVP acceptance criteria, interfaces, risk review, repository setup, initial evaluation set, backlog, and readiness gate.

2. Product concept and problem thesis

The platform should answer questions about the current state of organizational knowledge, historical versions, conflicting or superseded guidance, source authority, and - in a later focused extension - the provenance of an organizational decision or process change.

The system is intentionally not a generic "chat with documents" application. Its technical identity is version-aware retrieval, authority-aware retrieval, conflict surfacing, corrective retrieval, and measurable evaluation.

```
Organizational knowledge
        ↓
Reliable current-state retrieval
        ↓
Version / authority reasoning
        ↓
Conflict detection
        ↓
Historical retrieval
        ↓
Decision provenance
```

3. Scope: Phase 1 and Phase 2

3.1 Phase 1 - Organizational Knowledge Intelligence

Phase 1 is the foundation and must be independently portfolio-worthy before cloud polish or the full decision-intelligence extension.

- Ingest a real public organizational corpus with provenance and version metadata.
- Provide structure-aware chunking and dense + lexical retrieval.
- Use hybrid fusion and cross-encoder reranking.
- Handle current vs. historical queries explicitly.
- Model source authority and temporal validity.
- Detect and surface conflicting/superseded information.
- Use grounded generation with precise citations and refusal when evidence is insufficient.
- Use a bounded LangGraph workflow for routing, grading, and corrective retrieval.
- Evaluate against a fixed golden set and show ablation results versus a dense-only baseline.

3.2 Phase 2 - Focused Decision Intelligence

Add one focused capability: reconstruct why a documented organizational decision or process changed and how it evolved over time. Do not expand this into a general meeting assistant or an enterprise graph platform.

```
Initial problem
      ↓
Discussion
      ↓
Proposal/change artifact
      ↓
Decision/acceptance
      ↓
Implementation/process change
      ↓
Current documentation
```

A small but real Phase-2 decision-provenance MVP should be implemented before major AWS/frontend polish, because it is part of the product's unique technical thesis.

4. Primary user outcomes and query classes

Primary user: an engineer, technical employee, analyst, or knowledge worker who needs authoritative, current, and historically grounded information from a large organizational knowledge base.

Query mode	Example	Retrieval behavior
CURRENT	What is the current process for X?	Prioritize active/current, authoritative sources; suppress stale material when appropriate.
HISTORY	What was the previous process?	Retrieve historical/superseded versions from the requested period.
CHANGE	How has X changed?	Retrieve old and new versions plus transition artifacts.
WHY	Why did X change?	Broaden to current docs, historical versions, proposals, discussions, and change history.
UNANSWERABLE	What is the private internal reason for X?	Return insufficient evidence rather than fabricating an answer.

5. Corpus and data strategy

5.1 Use a real public corpus, not a synthetic enterprise

The original synthetic-company idea was rejected as the primary dataset. A synthetic corpus would make retrieval difficulty, conflicts, and evaluation partly circular because the project author would create the same problems the system is evaluated on.

The chosen organization should provide naturally messy, versioned, heterogeneous knowledge and real decision artifacts.

5.2 Leading corpus candidates

Candidate	Strength	Tradeoff	Current status
GitLab	Best enterprise/workplace resemblance; large public handbook and real Git history.	Decision rationale may be less formally structured than Kubernetes.	Current leading candidate.
Kubernetes	Excellent formal decision chains through KEPs, SIGs,	Feels more like a technical organization than a normal	Strong fallback / possible winner after validation.

	releases, deprecations, and docs.	enterprise.	
Python	Clean PEP decision process.	Less enterprise-like.	Not primary candidate.
Rust	Strong RFC history.	Less enterprise-like.	Not primary candidate.

5.3 Corpus validation gate

Before significant implementation, manually trace 3-5 real changes for both GitLab and Kubernetes. For each change, attempt to locate:

1. Original/earlier state
2. Discussion or identified problem
3. Proposal/change artifact
4. Decision or accepted change
5. Implementation/release/change
6. Current documentation

Choose the corpus only after verifying that these chains can be found reliably. This is Development Task #1, not another open-ended architecture exercise.

6. Open-source positioning

Using GitLab/Kubernetes public artifacts as the corpus is open-source/public-data usage, not automatically an upstream open-source contribution. The RAG system will live in its own repository and must not imply that the chosen organization commissioned, endorsed, or deploys the system.

- Publish the RAG system itself as an open-source portfolio project if desired.
- An upstream contribution is optional and only arises if a genuine issue is found in a dependency or source project.
- Respect the license and attribution requirements of the selected public corpus.

7. Engineering principles and 80/20 priorities

Guiding rule

Spend AI complexity on retrieval, provenance, temporal reasoning, conflict handling, and evaluation. Spend enough infrastructure and product complexity to prove that the AI system can be operated as real software. Keep every other layer as simple as the problem allows.

Priority	Area	Reason
1	Retrieval + ranking	Largest direct effect on answer quality and most important RAG hiring signal.
2	Metadata/version/provenance model	Defines current vs. historical truth and source authority.
3	Evaluation	Proves that advanced techniques actually improve the system.
4	Query routing / agentic orchestration	Makes retrieval adaptive rather than static.
5	Temporal reasoning + conflict detection	Differentiates the project from generic RAG.
6	Backend/cloud productionization	Demonstrates general SWE ability once the retrieval thesis works.

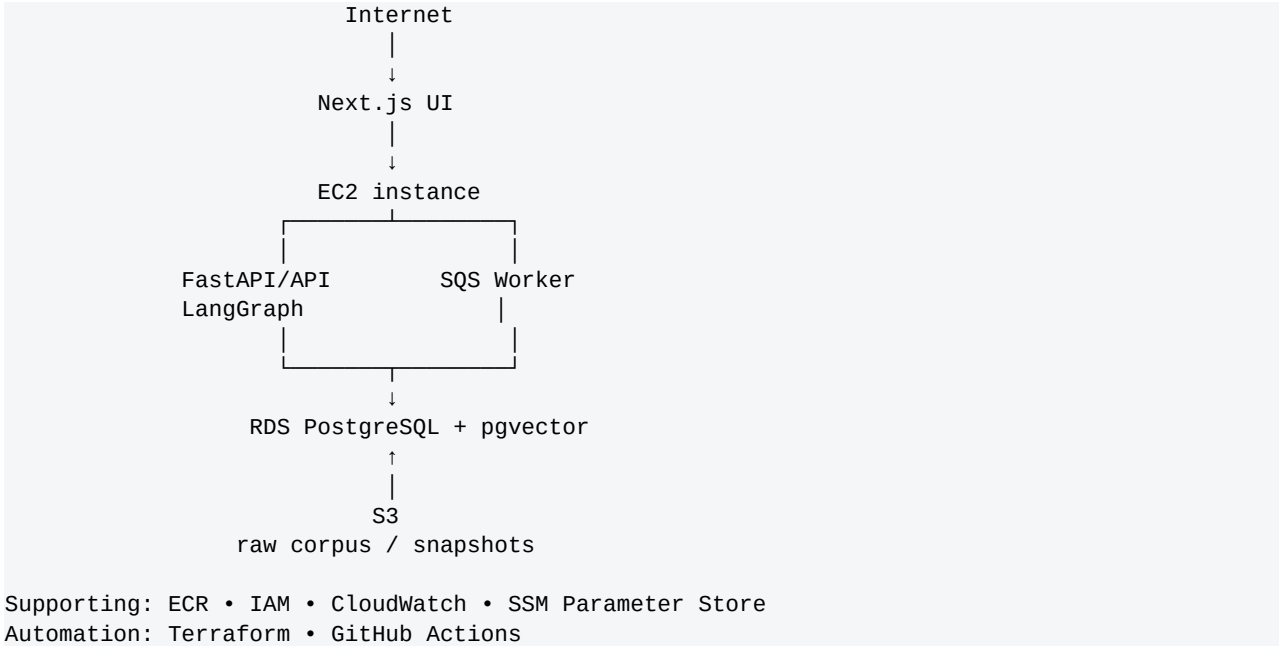
7	Chunking/UI polish	Important, but should not consume disproportionate effort.

8. Target system architecture

8.1 Local development architecture

```
Public corpus / Git
       ↓
Ingestion + version metadata
       ↓
Markdown/HTML/PDF parsing
       ↓
Structure-aware chunks
       ↓
PostgreSQL + pgvector + FTS
       ↑
Dense retrieval + lexical retrieval
       ↓
RRF + temporal/authority adjustments
       ↓
Cross-encoder reranker
       ↓
Conflict detection + evidence grading
       ↓
LangGraph corrective retrieval
       ↓
Grounded answer + citations
       ↓
FastAPI → Streamlit (development) / Next.js (final)
```

8.2 AWS portfolio architecture



Supporting: ECR • IAM • CloudWatch • SSM Parameter Store
Automation: Terraform • GitHub Actions

EC2 is the chosen baseline compute option (Option B). ECS/Fargate + ALB are not required for the permanent portfolio deployment and should only be revisited as a temporary learning extension if desired.

9. Technology stack

Layer	Chosen technology	Reason
Language/runtime	Python 3.11+	Primary ecosystem for RAG, FastAPI, LangGraph, evaluation, and retrieval tooling.
Parsing	Git/API + Markdown/HTML parsers; Docling for PDFs only	Match parser to source type; avoid over-processing clean Markdown.
Chunking	Structure-aware + deterministic provenance context	Preserves real hierarchy and metadata without unnecessary LLM calls.
Embeddings	Voyage 4 Lite initially	Modern retrieval model, low cost, easily swappable for experiments.
Database	PostgreSQL + pgvector + native full-text search	One system for relational metadata, vectors, lexical retrieval, and relationships.
Retrieval	Dense + lexical → RRF → temporal/authority scoring → BGE reranker	Advanced but justified retrieval pipeline.
Agent orchestration	LangGraph	Readable bounded state graph for routing, grading, retry, and decomposition.
Generation	Frontier hosted LLM with structured output	Grounded synthesis, citation objects, conflict/history fields.
Evaluation	Golden set + standard IR metrics + Ragas where useful + custom metrics	Separates retrieval, generation, and agent trajectory performance.
Tracing	Langfuse	Inspect routing, retrieval, reranking, retries, latency, and cost.
Backend	FastAPI	Production-relevant Python API layer.
Dev UI	Streamlit	Fast experimentation while retrieval is changing.
Final UI	Next.js / React	Portfolio-quality interface for evidence, conflicts, and timelines.
Cloud	AWS: EC2, RDS, S3, SQS, IAM, CloudWatch, ECR, SSM	Core services directly justified by the product.
IaC / CI	Terraform + GitHub Actions	Reproducible infrastructure and automated verification/deployment.

10. Ingestion and parsing

Git-native ingestion is more important than sophisticated PDF parsing for the likely corpus. Preserve raw source artifacts and provenance so the index can be reproduced.

- **Primary sources:** Git repositories, public APIs, Markdown, front matter, HTML.
- **Secondary parser:** Docling only for PDFs or genuinely messy documents.
- **Preserve:** repository, file path, source URL, commit SHA, commit date, version, source type, first-seen date, last-modified date, status, and authority tier.
- **Incremental ingestion:** detect changed/new documents and avoid blindly rebuilding the entire corpus when possible.

11. Chunking and enrichment

Use document structure and heading hierarchy as the primary chunking signal. Avoid splitting tables or coherent sections unnecessarily. Exact token targets should be tuned against retrieval metrics rather than treated as fixed requirements.

```
Organization: GitLab
Source type: Official Handbook
Section: Product > Development > Launch Criteria
Status: Current
Authority: Official
Document date: ...
Commit: ...

<chunk content>
```

LLM-generated contextual chunk enrichment is optional and should be tested later as an experiment. Deterministic metadata context is the baseline because it is cheap, reproducible, and directly relevant to temporal/authority retrieval.

12. Embeddings

Start with Voyage 4 Lite behind a clean embedding interface/configuration. Do not select a more expensive embedding model until the evaluation harness demonstrates a meaningful benefit. Re-indexing should be possible from configuration rather than requiring code changes.

13. PostgreSQL/pgvector data model

The relational schema is central because temporal validity, authority, provenance, and relationships are structured data problems as much as retrieval problems.

```
documents
---------
id
canonical_id
organization
title
source_type
source_url
authority_tier
status

document_versions
-----------------
id
document_id
version
commit_sha
valid_from
valid_until
created_at
modified_at
superseded_by

chunks
------
id
document_version_id
heading_path
content
```

```
embedding
search_vector
position

document_relationships
----------------------
from_document_id
to_document_id
relationship_type
confidence
source
```

Potential Phase-2 additions: entities, entity_mentions, decisions, decision_evidence. Start with PostgreSQL joins and recursive CTEs. Do not introduce Neo4j unless a real traversal limitation appears.

14. Retrieval architecture

```
Query
  ↓
Query-mode classification
  ↓
Metadata constraints
  ↓
┌──────────────────────┐
│ Lexical retrieval    │
│ Dense retrieval      │
└──────────┬───────────┘
           ↓
Reciprocal Rank Fusion
           ↓
Temporal + authority adjustment
           ↓
Candidate set (~30-50 initially)
           ↓
BGE cross-encoder reranking
           ↓
Top evidence (~5-10 initially)
```

PostgreSQL full-text search should be described accurately as lexical/full-text retrieval, not BM25 unless an actual BM25 implementation is introduced. Exact candidate counts, RRF constants, and weights are hypotheses to tune against the golden set.

15. Temporal and authority-aware retrieval

Temporal and authority metadata should influence candidate selection before final generation, not merely be post-processing performed by the LLM.

- CURRENT queries strongly prefer active/current, official, authoritative documents.
- HISTORY queries intentionally include superseded versions valid during the requested period.
- CHANGE queries retrieve both previous and current state plus transition artifacts.
- WHY queries broaden to lower-authority discussion/proposal sources because rationale may live outside official current documentation.

The authority hierarchy must be derived from the chosen corpus rather than guessed globally. Example concept: current official docs > accepted proposal > official release/change record > maintainer discussion > issue/community discussion.

16. Conflict detection

Conflict detection is a first-class component. Do not simply pass contradictory evidence into the generator and hope the model resolves it correctly.

```
Current authoritative position: ...
Superseded position: ...
Historical proposal: ...
Conflict/change type: temporal supersession
Resolution status: resolved / unresolved
```

- Conflict categories may include current vs. stale, official vs. informal, proposal vs. accepted decision, temporal supersession, and genuinely unresolved contradiction.
- Unresolved authoritative conflicts should be surfaced rather than silently resolved.

17. Agentic orchestration

Use LangGraph for a small bounded workflow. The project does not need multi-agent swarms.

```
Query
  ↓
Query-mode router
  ↓
Retrieval plan
  ↓
Retrieve → rerank → conflict detection
  ↓
Evidence grader
  ↓
Enough evidence?
  ├─ Yes → generate answer
  └─ No  → rewrite/decompose/change filters → retry

max_iterations = 2-3
```

Corrective retrieval remains only if it measurably improves difficult-query performance enough to justify additional latency/cost.

18. Generation, citations, and refusal

The generator receives structured provenance rather than being asked to infer it. Use structured output for answer status, evidence, conflicts, historical context, and insufficient-evidence reasons.

```
answer
answer_status
evidence[]
conflicts[]
historical_context[]
confidence
insufficient_evidence_reason
```

- Citations should expose document/chunk identity, source URL, supporting span where practical, authority/status, and document date/version.
- Refuse or return insufficient evidence when the corpus does not support the requested claim.
- Do not invent private organizational rationale that is absent from the public corpus.

19. Evaluation and experimentation

19.1 Golden set

Create the first 30-50 questions soon after a small corpus is indexed, before repeatedly tuning retrieval. Grow toward roughly 75-100 questions later.

Category	Initial target
Basic factual	~8
Cross-document	~6
Current-state	~8
Historical/version	~6
Conflict/authority	~5
Unanswerable	~5

Record question, expected facts, expected source documents/chunks where practical, query class, and whether the question is answerable.

19.2 Metrics

- **Retrieval:** Recall@K, MRR, NDCG; latency where useful.
- **Generation:** faithfulness/groundedness, answer relevance, citation correctness, refusal correctness.
- **Project-specific:** current-document accuracy, stale-result rate, authority-selection accuracy, temporal retrieval accuracy, conflict precision/recall, router accuracy.
- **Agent trajectory:** grader accuracy, retry frequency, iteration count, selected retrieval strategy, latency, cost/query.
- **Phase 2:** decision-chain completeness.

19.3 Ablation table

Evaluate the same fixed dataset after each meaningful addition. The README should show a progression such as:

Pipeline	Expected purpose
Dense baseline	Establish a deliberately simple reference point.
+ lexical + RRF	Test hybrid recall and exact-term retrieval.
+ reranker	Test precision improvement.
+ temporal/authority metadata	Improve current-state and historical correctness.
+ conflict handling	Improve trustworthy answers over inconsistent evidence.
+ corrective agentic retrieval	Test whether retries improve hard-query performance enough to justify cost/latency.

Portfolio artifact

The metrics/ablation table, evaluation set, traces, and explanation of techniques that did or did not help are at least as important as the chat UI.

20. Observability

Use Langfuse for AI workflow traces and CloudWatch for deployed application/infrastructure logs and basic metrics. Do not build a large observability platform.

- Langfuse: query mode, filters, retrieved candidates/scores, reranker scores, conflicts, grader result, retries, model cost/latency.
- CloudWatch: FastAPI/worker logs, ingestion failures, basic host/application metrics, a small number of alarms.

21. Backend engineering and APIs

The backend should demonstrate normal production software practices in addition to AI behavior. Implement only the subset that materially improves robustness and interview signal.

Endpoint / concern	Purpose
POST /query	Run the grounded RAG workflow.
POST /ingestion-jobs	Create asynchronous ingestion/indexing job.
GET /ingestion-jobs/{id}	Inspect job status/progress/failure.
GET /documents/{id}	Inspect source document.
GET /documents/{id}/versions	Inspect version history.
GET /sources/{id}	Resolve evidence/source metadata.
GET /health	Liveness.
GET /ready	Readiness/dependency status.

- Request validation and typed contracts.
- Centralized exception handling.
- Structured logging and request IDs.
- Database migrations and connection pooling.
- Timeouts/retry policies for external model APIs.
- Idempotent ingestion operations.
- API versioning only if it becomes useful; do not add ceremony prematurely.

22. Asynchronous ingestion

Ingestion is a justified asynchronous workflow because cloning/fetching, parsing, chunking, embedding, and indexing can take minutes and should not hold an HTTP request open.

```
POST /ingestion-jobs
        ↓
Create job record
        ↓
       SQS
        ↓
Worker on EC2
        ↓
fetch → parse → version → chunk → embed → index
        ↓
RDS + S3

GET /ingestion-jobs/{job_id} → status/progress
```

SQS Standard is appropriate for the portfolio workload. Worker logic must be idempotent and tolerate retries/duplicate delivery. Track partial failure and job progress in PostgreSQL.

23. AWS deployment architecture

23.1 Core AWS services - chosen

Service	Role	Decision
EC2	Run FastAPI and ingestion worker containers.	Chosen baseline compute (Option B).
RDS PostgreSQL + pgvector	Managed RAG database.	Keep; central paid managed service.
S3	Raw corpus, snapshots, parsed artifacts/evaluation outputs where useful.	Keep; natural fit and very low cost.
SQS	Async ingestion jobs.	Keep; real distributed-systems value at negligible cost.
IAM	Service-to-service permissions.	Keep; foundational AWS skill.
CloudWatch	Application logs + basic monitoring.	Keep; low-complexity operational visibility.
ECR	Container image registry.	Keep because deployment is containerized.
SSM Parameter Store	Configuration/API secrets.	Keep; use instead of Secrets Manager for this project.
Terraform	Provision AWS resources reproducibly.	Required.
GitHub Actions	CI/CD and deployment automation.	Required.

23.2 Services intentionally spared

Service	Decision	Reason
NAT Gateway	Do not use	Cost disproportionate to a portfolio workload.
ECS/Fargate + ALB	Not baseline	EC2 provides enough AWS/compute signal at lower recurring cost. Optional short-lived extension only.
Secrets Manager	Skip	SSM Parameter Store is sufficient; no need for rotation lifecycle features.
Cognito / fine-grained RBAC	Optional/deferred	Public corpus means elaborate permission boundaries would be artificial.
CloudFront / Route 53	Polish only	Not required for the technical core.
Lambda / API Gateway	Skip	Long-running/streaming RAG and local reranker/worker model fit EC2 better.
ElastiCache / OpenSearch / DynamoDB	Skip	No demonstrated requirement.
Neo4j	Skip initially	PostgreSQL relationships and recursive CTEs are enough at this scale.
Kafka / microservices / Kubernetes	Skip	No workload or team-scale requirement justifies them.

23.3 Environment strategy

Environment	Shape
Local development	Docker Compose: PostgreSQL/pgvector, FastAPI, worker, Langfuse, Streamlit.
CI/test	GitHub Actions with tests, small RAG regression set, and Docker build.
AWS portfolio	EC2 + RDS + S3 + SQS + ECR + IAM + CloudWatch + SSM, provisioned by Terraform.

24. CI/CD and Infrastructure as Code

Productionization should strengthen the SWE story without changing the core RAG design.

```
Pull request
  ↓
format/lint/typecheck
  ↓
unit + integration tests
  ↓
small RAG smoke evaluation (~10-15 queries)
  ↓
Docker build

Main branch
  ↓
build/push image to ECR
  ↓
deploy EC2 containers / application update

Manual or scheduled
  ↓
full 75-100 question RAG benchmark
```

Terraform should define the AWS portfolio environment so it can be recreated rather than manually assembled in the console. Infrastructure complexity should remain limited to what the project actually uses.

25. Frontend roadmap

25.1 Development UI

Use Streamlit while retrieval and evaluation are changing. UI polish has low value during RAG experimentation.

25.2 Final portfolio UI

Once the AI pipeline is stable, build a Next.js/React interface. The frontend should expose the system's differentiators rather than becoming a generic chat screen.

- Answer with citation/source panel.
- Source authority/status/version display.
- Conflict panel showing old/current/unresolved information.
- Historical timeline.
- Phase-2 decision-provenance visualization.
- Ingestion-job status/admin view if useful.

Authentication can be basic or omitted initially. Fine-grained document RBAC is deferred unless a real requirement emerges.

26. Testing and production hardening

26.1 Ordinary software testing

- **Unit tests:** chunking, metadata extraction, RRF, authority calculations, temporal filtering, query classification, conflict logic, idempotency helpers.
- **Integration tests:** ingestion → database, lexical/vector retrieval, SQS → worker, FastAPI → PostgreSQL, model API failure behavior.
- **E2E/smoke:** one small user query path plus ingestion job path in a test environment where practical.

26.2 Production-minded behaviors to implement

- Structured logs and request IDs.
- Health/readiness endpoints.
- Timeouts and bounded retries.
- Database migrations and connection pooling.
- Idempotent ingestion and duplicate SQS handling.
- Safe secret/config handling.
- Basic rate/cost protections for AI endpoints if exposed publicly.

Do not implement every enterprise feature. Six to eight production practices done properly are more valuable than a long checklist implemented superficially.

27. Cost and operational constraints

Hard project constraint
Target recurring AWS cost: <= US$50/month for the always-on portfolio deployment. Prefer local development; AWS is the deployment/operations environment, not the everyday experimentation environment.

- Use a small EC2 instance rather than permanent ECS/Fargate + ALB.
- Use a small single-AZ RDS instance appropriate for the corpus size.
- Keep S3/SQS/ECR/CloudWatch usage small and intentional.
- Avoid NAT Gateway.
- Use SSM Parameter Store instead of paid secret-management features that are not needed.
- Tear down nonessential resources when not demoing if costs rise.

28. Explicit non-goals and deferred technologies

The following are not required for the core project unless a measured requirement appears:

- Multi-agent swarms or supervisor architectures.
- Neo4j/Graph database.
- Pinecone or a separate vector database.
- Elasticsearch/OpenSearch.
- Kafka or microservices.
- Kubernetes deployment.
- LLM fine-tuning or custom embedding training.
- Multi-region/high-availability production architecture.

- Fine-grained enterprise RBAC over the public corpus.
- Elaborate authentication/account recovery flows.
- Complex frontend before retrieval stabilizes.

29. Development roadmap

Stage	Deliverable
0. Corpus validation	Compare GitLab vs. Kubernetes; manually trace 3-5 real changes; choose corpus.
1. Repository + local foundation	Docker Compose, Postgres/pgvector, migrations, config, minimal FastAPI skeleton.
2. Ingestion + dense baseline	Ingest small real corpus subset; structure-aware chunks; Voyage embeddings; dense retrieval; basic grounded answer.
3. Initial golden set	Create 30-50 fixed evaluation questions and baseline metrics.
4. Hybrid retrieval	Add PostgreSQL lexical search + RRF; re-evaluate.
5. Reranking	Add local BGE cross-encoder; re-evaluate.
6. Project-specific metadata retrieval	Implement query modes, temporal/version logic, authority handling; re-evaluate.
7. Conflict detection	Add first-class conflict component and metrics.
8. Agentic retrieval	Add LangGraph routing, evidence grading, bounded corrective retrieval; measure cost/benefit.
9. Phase-2 provenance MVP	Implement one real decision/change chain end-to-end.
10. Backend hardening + async ingestion	SQS-style local abstraction/job model, idempotency, logs, health/readiness, tests.
11. AWS portfolio deployment	EC2, RDS, S3, SQS, ECR, IAM, CloudWatch, SSM via Terraform.
12. CI/CD	GitHub Actions, ECR build/push, deployment automation, smoke RAG eval.
13. Final frontend	Replace Streamlit with Next.js; evidence/conflict/timeline UI.
14. Final evaluation + portfolio assets	Full benchmark, architecture docs, README, demo video, measured resume bullets.

30. Definition of done

30.1 Phase 1 done

- Real public corpus ingestion with provenance and versions.
- Incremental/version-aware indexing.
- Dense + lexical hybrid retrieval.
- Cross-encoder reranking.
- Temporal/current-vs-historical behavior.
- Source authority handling.
- Conflict detection.
- Grounded answers with citations.
- Correct refusal for insufficient evidence.
- Bounded LangGraph corrective retrieval.
- Fixed golden-set evaluation.
- Measured improvement over dense-only baseline.

- FastAPI backend and traces/observability.

AWS, Next.js, and the full Phase-2 decision feature are not required to declare the RAG foundation complete.

30.2 Whole project done

- At least one real decision-provenance chain reconstructed end-to-end.
- Asynchronous ingestion job workflow.
- AWS deployment using the chosen core services.
- Terraform and GitHub Actions CI/CD.
- Next.js portfolio UI exposing evidence, conflicts, and timeline/provenance.
- Architecture documentation, evaluation report, README, and short demo video.

31. Portfolio and interview positioning

Do not describe the work as an official RAG system for GitLab/Kubernetes. Frame it as an independent organizational knowledge platform evaluated on a real public organizational corpus.

Core interview story

Enterprise RAG systems have a harder problem than semantic similarity: knowledge changes over time, sources have different authority, and documents conflict. I built a retrieval system that models those properties explicitly, evaluated each retrieval improvement against a fixed corpus, and then productionized the system as a cost-conscious AWS application.

Potential final resume framing (replace generic wording with measured results once available):

- Built an advanced organizational knowledge RAG platform over real-world versioned public documentation using hybrid lexical/vector retrieval, cross-encoder reranking, temporal/authority-aware ranking, explicit conflict detection, and source-grounded generation.
- Designed a provenance-aware ingestion and retrieval model in PostgreSQL/pgvector for document versions, supersession relationships, source authority, and current-vs-historical state.
- Deployed a containerized FastAPI RAG service on AWS using EC2, RDS, S3, SQS, ECR, IAM, CloudWatch, and SSM, provisioned with Terraform and delivered through GitHub Actions.
- Implemented and benchmarked retrieval/agent improvements with a fixed golden set and custom metrics for stale-document rate, temporal accuracy, authority selection, conflicts, and decision-chain completeness.

32. Remaining planning/readiness gaps

Architecture planning is substantially complete. The remaining items are bounded pre-build/readiness tasks. They should be resolved while starting the repository rather than through more open-ended architecture brainstorming.

Gap / gate	What remains	Blocking development?
1. Final corpus selection	Run the GitLab vs. Kubernetes validation exercise and choose one.	Yes - first task.
2. MVP product spec	Freeze the smallest complete Phase-1 user outcome, must-haves, non-goals, and testable acceptance criteria.	Yes for disciplined implementation.
3. Domain/data model review	Finalize initial entities/constraints after the corpus structure is known; especially versions, authority, status, and ingestion jobs.	Before migrations stabilize.
4. Authority/status taxonomy	Define corpus-specific source types and precedence rules after corpus selection.	Before temporal/authority retrieval.

5. Initial golden set	Create ~30-50 questions after a representative subset is indexed and before retrieval tuning.	Before advanced optimization.
6. Risk/security/cost review	Document model-API abuse/cost risk, secret handling, prompt/data injection concerns, public endpoint exposure, dependency risk, and AWS cost guardrails.	Before public deployment.
7. Interface contracts	Define API/job inputs, outputs, error shapes, idempotency/retry behavior, and dependency failure policy.	Before major backend work.
8. Repository startup docs	Create README, product-spec, architecture, domain model, interfaces, decisions log, AGENTS.md/CLAUDE.md as appropriate.	Before coding-agent-heavy development.
9. Environment/config plan	Define local/test/AWS env vars, SSM parameters, migration tooling, and reproducible setup.	Before CI/deployment.
10. Quality gates	Lint/format/typecheck/tests/build commands must work on the skeleton.	Before feature implementation expands.
11. CI + minimal deployment path	Green CI and a verified skeleton deployment path before the project becomes large.	Before AWS productionization.
12. Bounded backlog	Convert roadmap into small GitHub issues with acceptance criteria and dependencies.	Before sustained implementation.
13. First vertical slice	Plan the smallest end-to-end outcome: ingest a small real subset → retrieve → answer with citation → evaluate.	Yes - immediate next implementation target.
14. Readiness review	Run a final pass over requirements, model, architecture, risks, interfaces, repo, tests, CI, and first slice.	Final gate before sustained feature work.

Immediate next action

Stop expanding the architecture. Validate the corpus, freeze the MVP acceptance criteria, create the repository/local stack, ingest a small real subset, build the deliberately simple dense baseline, and write the first golden evaluation set.

Appendix A - Current decisions at a glance

Decision	Current choice
Product	Organizational Knowledge + Decision Intelligence RAG
Phase strategy	Phase 1 knowledge intelligence; Phase 2 focused decision provenance
Corpus	GitLab vs. Kubernetes pending validation
Database	PostgreSQL + pgvector + native FTS
Embeddings	Voyage 4 Lite initially
Reranker	Local BGE cross-encoder
Agent orchestration	LangGraph, bounded single workflow
Backend	FastAPI
Development UI	Streamlit
Final UI	Next.js / React

Observability	Langfuse + CloudWatch
AWS compute	EC2 baseline
AWS data	RDS + S3
AWS async	SQS
AWS ops	IAM + CloudWatch + ECR + SSM Parameter Store
IaC / CI	Terraform + GitHub Actions
Recurring AWS cost target	<= US$50/month
Infrastructure stance	No service without a concrete requirement; avoid NAT Gateway and permanent ECS/ALB by default.